

Python express

Les bases de Python nécessaires pour construire notre Assistant IA d'Entreprise

Cours pas à pas · Code expliqué ligne par ligne · Commandes et résultats · Exercices corrigés

Sommaire

Partie	Contenu
1	Position dans la formation et méthode de lecture
2	Les outils du projet et le dossier fil rouge
3	Installation de l'environnement : Python, VS Code, venv, pip
4	Premier programme
5	Variables et types
6	Les listes
7	Les dictionnaires (et le format JSON)
8	Les conditions
9	Les boucles
10	Les fonctions
11	Lire un fichier
12	Modules, API et gestion des erreurs
13	Mini-projet : le mini-assistant RH (version 0 du projet)
14	Exercices corrigés
15	Récapitulatif et glossaire

PARTIE 1

Position dans la formation

Élément	Détail
Jour	Jour 1 — après-midi (après le cours IA / LLM / prompt engineering)
Durée	1 h à 1 h 15 (cours + exercices). Le profil avancé peut aller directement au mini-projet (partie 13).
Prérequis	Aucun. Avoir un ordinateur avec droits d'installation.
Place dans le projet	C'est la fondation : toutes les notions vues ici seront réutilisées dans <code>llm.py</code> , <code>rag.py</code> , <code>agent.py</code> et <code>main.py</code> .

Objectifs

- Installer et préparer l'environnement de travail (Python, VS Code, environnement virtuel)
- Écrire et exécuter un programme Python
- Maîtriser **uniquement** les notions utiles au projet : variables, listes, dictionnaires, conditions, boucles, fonctions, fichiers, modules, gestion des erreurs
- Construire un **mini-assistant sans IA** : la version 0 de notre Assistant IA d'Entreprise

Principe de ce document

On n'apprend pas « tout Python ». On apprend **le Python nécessaire pour construire un assistant IA**. Chaque notion est reliée à un usage précis dans le projet.

Comment lire chaque exemple ?

Chaque notion suit toujours le même schéma :

1. **Le code** — dans un bloc gris avec les numéros de ligne (ne recopiez pas les numéros !)
2. **L'explication ligne par ligne** — dans un tableau
3. **La commande** pour exécuter le programme
4. **Le résultat obtenu** — ce que vous devez voir à l'écran
5. **Le lien avec le projet** — à quoi cette notion va nous servir

PARTIE 2

Les outils du projet

Voici tous les outils que nous allons utiliser pendant les 3 jours. **Aujourd'hui, nous installons seulement les 4 premiers** ; les autres seront installés au moment où on en aura besoin.

Outil	Rôle dans le projet	Analogie	Quand ?
Python 3.12	Le langage de programmation du projet	La langue que l'on parle à l'ordinateur	Jour 1 → 3
VS Code	Éditeur pour écrire le code et lancer le terminal	Le cahier et le stylo	Jour 1 → 3
Terminal (PowerShell)	Exécuter des commandes et des programmes	Le tableau de bord de l'ordinateur	Jour 1 → 3
pip + venv	Installer des bibliothèques dans un environnement isolé	Une boîte à outils rangée par projet	Jour 1 → 3
requests	Appeler une API (Ollama, puis notre FastAPI)	Le facteur qui envoie et rapporte les lettres	Jour 1
pandas, scikit-learn	Créer notre premier modèle de Machine Learning (TP 1)	Un atelier pour fabriquer un modèle	Jour 1
Ollama + Llama 3.2 / Phi-3	Faire tourner un LLM en local , sans Internet	Un ChatGPT installé sur votre PC	Jour 1 → 3
pypdf, python-docx	Lire les documents PDF et Word de l'entreprise	Les yeux de l'assistant	Jour 2
sentence-transformers	Transformer le texte en vecteurs (embeddings)	L'adresse GPS du sens des phrases	Jour 2
ChromaDB	Base de données vectorielle pour le RAG	La bibliothèque où l'on range les documents	Jour 2
LangGraph	Construire l'agent (nœuds, routage, mémoire)	Le chef d'orchestre	Jour 2
FastAPI + Uvicorn	Exposer l'assistant sous forme d'API	Le serveur du restaurant	Jour 3
Streamlit	Interface de chat web en quelques lignes	La vitrine du magasin	Jour 3
Docker Desktop	Emballer l'application dans un conteneur	Un carton de déménagement prêt à partir	Jour 3
n8n	Automatiser : formulaire → assistant → e-mail	Un robot qui relie les applications	Jour 3
Git / GitHub (bonus)	Sauvegarder et partager le code	L'historique des versions	Jour 3

Le dossier du projet fil rouge

Tout le projet est construit dans **un seul dossier**. Chaque jour ajoute un fichier qui **réutilise le précédent** :

ARBORESCENCE DU PROJET

```
Projet_Assistant_IA/
├── venv/                ← environnement virtuel (créé aujourd'hui)
├── documents/          ← documents de l'entreprise (rh.txt aujourd'hui)
│   ├── rh.txt
│   ├── procedure.pdf   ← Jour 2
│   └── formation.docx  ← Jour 2
├── exercices/         ← les petits programmes de ce document
├── mini_assistant.py   ← AUJOURD'HUI : version 0 (sans IA)
├── llm.py              ← Jour 1 (TP 3) : connexion à Ollama
├── rag.py              ← Jour 2 matin
├── agent.py            ← Jour 2 après-midi
├── main.py             ← Jour 3 (API FastAPI)
├── app.py              ← Jour 3 (interface Streamlit)
├── Dockerfile          ← Jour 3
└── requirements.txt    ← liste des bibliothèques
```

PARTIE 3

Installation de l'environnement

3.1 Installer Python

1. Allez sur python.org/downloads et téléchargez **Python 3.12** (version recommandée pour la compatibilité avec toutes les bibliothèques du projet).
2. Lancez l'installateur. **Très important** : cochez la case « **Add python.exe to PATH** » en bas de la première fenêtre.
3. Cliquez sur **Install Now** et attendez la fin de l'installation.

Attention

Si vous oubliez de cocher « **Add python.exe to PATH** », la commande `python` ne sera pas reconnue dans le terminal. Solution : relancez l'installateur → **Modify** → cochez **Add Python to environment variables**.

Vérifier l'installation

COMMANDE (TERMINAL)

```
python --version
```

Demande à Python d'afficher sa version. Si un numéro s'affiche, Python est bien installé et accessible depuis le terminal.

RÉSULTAT OBTENU

```
Python 3.12.x
```

Mac / Linux

Sur Mac et Linux, la commande s'appelle souvent `python3` au lieu de `python` (idem : `pip3` au lieu de `pip`).

3.2 Installer VS Code

1. Téléchargez VS Code sur code.visualstudio.com et installez-le (options par défaut).
2. Ouvrez VS Code → cliquez sur l'icône **Extensions** (4 carrés à gauche) ou `Ctrl + Shift + X`.
3. Cherchez **Python** (éditeur : Microsoft) et cliquez sur **Install**.

3.3 Créer le dossier du projet

1. Créez un dossier **Projet_Assistant_IA** (par exemple sur le Bureau).
2. Dans VS Code : **File** → **Open Folder...** → choisissez ce dossier.
3. Ouvrez le terminal intégré : menu **Terminal** → **New Terminal** (ou `Ctrl + ``).

À retenir

Le terminal s'ouvre **directement dans le dossier du projet**. Toutes les commandes de ce document se tapent dans ce terminal.

3.4 Créer l'environnement virtuel (venv)

Définition

Un **environnement virtuel** est une copie isolée de Python **propre à un projet**. Les bibliothèques installées pour ce projet ne se mélangent pas avec celles des autres projets.

Analogie simple

C'est comme une **boîte à outils réservée à un chantier** : ce que vous y mettez reste dans cette boîte et ne dérange pas les autres chantiers.

COMMANDE (TERMINAL)

```
python -m venv venv
```

python lance Python ; **-m venv** utilise le module « venv » ; le dernier **venv** est le nom du dossier créé. Un dossier `venv/` apparaît dans le projet.

Activer l'environnement**COMMANDES (TERMINAL)**

```
# Windows (PowerShell)
venv\Scripts\activate

# Mac / Linux
source venv/bin/activate
```

Active l'environnement virtuel. À partir de maintenant, `python` et `pip` utilisent la boîte à outils du projet.

RÉSULTAT OBTENU

```
(venv) PS C:\Users\...\Projet_Assistant_IA>
```

À retenir

Le préfixe **(venv)** au début de la ligne prouve que l'environnement est activé. **À refaire à chaque nouvelle ouverture du terminal.**

Erreur fréquente sous Windows

Sous Windows, si PowerShell affiche « *l'exécution de scripts est désactivée sur ce système* », tapez une seule fois :

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Répondez **O** (Oui), puis relancez la commande d'activation.

3.5 Installer une bibliothèque avec pip

Définition

pip est le gestionnaire de bibliothèques de Python. Il télécharge et installe des outils écrits par d'autres développeurs (requests, FastAPI, ChromaDB...).

COMMANDE (TERMINAL)

```
pip install requests
```

Télécharge et installe la bibliothèque **requests**, qui permet d'envoyer des requêtes HTTP (appeler une API). Nous l'utiliserons pour parler à Ollama.

RÉSULTAT OBTENU

```
Collecting requests
  Downloading requests-2.x.x-py3-none-any.whl
...
Successfully installed certifi-... charset-normalizer-... idna-... requests-2.x.x
urllib3-...
```

COMMANDE (TERMINAL)

```
pip list
```

Affiche toutes les bibliothèques installées dans l'environnement.

RÉSULTAT OBTENU

```
Package          Version
-----
certifi          ...
requests         2.x.x
...
```

COMMANDE (TERMINAL)

```
pip freeze > requirements.txt
```

Écrit la liste exacte des bibliothèques (avec leurs versions) dans le fichier **requirements.txt**. Ce fichier permet à n'importe qui de réinstaller le projet avec `pip install -r requirements.txt`. Il servira aussi au Dockerfile le jour 3.

3.6 Récapitulatif des commandes

Commande	À quoi elle sert
<code>python --version</code>	Vérifier que Python est installé
<code>python -m venv venv</code>	Créer l'environnement virtuel (une seule fois)
<code>venv\Scripts\activate</code>	Activer l'environnement (à chaque ouverture du terminal)
<code>pip install <nom></code>	Installer une bibliothèque
<code>pip list</code>	Voir les bibliothèques installées
<code>pip freeze > requirements.txt</code>	Sauvegarder la liste des bibliothèques
<code>pip install -r requirements.txt</code>	Réinstaller toutes les bibliothèques d'un projet
<code>python fichier.py</code>	Exécuter un programme Python
<code>deactivate</code>	Désactiver l'environnement virtuel

PARTIE 4

Premier programme

Dans VS Code, créez un dossier **exercices**, puis un fichier **exercices/hello.py** et tapez :

FICHIER : `exercices/hello.py`

```
1 # Mon premier programme Python
2 print("Bonjour, bienvenue dans la formation IA !")
3 print("Nous allons construire un assistant IA.")
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code># Mon premier programme Python</code>	Un commentaire : tout ce qui suit <code>#</code> est ignoré par Python. Il sert à expliquer le code aux humains.
2	<code>print("Bonjour, bienvenue dans la formation IA !")</code>	print() affiche à l'écran le texte placé entre guillemets.
3	<code>print("Nous allons construire un assistant IA.")</code>	Deuxième affichage. Python exécute les lignes de haut en bas , une par une.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python exercices/hello.py
```

RÉSULTAT OBTENU

```
Bonjour, bienvenue dans la formation IA !
Nous allons construire un assistant IA.
```

Attention

Enregistrez toujours le fichier (**Ctrl + S**) avant de l'exécuter, sinon Python exécute l'ancienne version. Si vous voyez « *No such file or directory* », vérifiez que le terminal est bien dans le dossier **Projet_Assistant_IA**.

PARTIE 5

Variables et types

Définition

Une **variable** est une **boîte avec une étiquette** qui contient une valeur. On la crée avec le signe `=` : à gauche le nom, à droite la valeur.

Type	Nom Python	Exemple	Usage dans le projet
Texte	<code>str</code>	"Bonjour"	Question de l'utilisateur, réponse du LLM
Nombre entier	<code>int</code>	30	Nombre de documents, numéro d'étape
Nombre décimal	<code>float</code>	0.7	Température du modèle
Booléen	<code>bool</code>	True / False	Le modèle a-t-il fini de répondre ?

FICHIER : exercices/variables.py

```

1 nom = "Assistant IA"
2 version = 1
3 temperature = 0.7
4 est_actif = True
5
6 print(nom)
7 print(type(nom))
8 print(type(version))
9 print(type(temperature))
10 print(type(est_actif))
11 print(f"{nom} version {version} - température : {temperature}")

```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>nom = "Assistant IA"</code>	Crée la variable nom contenant un texte (<code>str</code>). Le texte est toujours entre guillemets.
2	<code>version = 1</code>	Variable version contenant un nombre entier (<code>int</code>), sans guillemets.
3	<code>temperature = 0.7</code>	Nombre décimal (<code>float</code>) : on utilise un point , pas une virgule.
4	<code>est_actif = True</code>	Booléen (<code>bool</code>) : True ou False, avec une majuscule.
6	<code>print(nom)</code>	Affiche le contenu de la variable (sans guillemets).
7	<code>print(type(nom))</code>	type() indique le type de la variable.
11	<code>print(f"{nom} version {version} - température : {temperature}")</code>	Une f-string : le f devant les guillemets permet d'insérer des variables entre accolades <code>{ }</code> . Très utilisée pour construire les prompts.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python exercices/variables.py
```

RÉSULTAT OBTENU

```
Assistant IA
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
Assistant IA version 1 - température : 0.7
```

Lien avec le projet

Dans le projet, on construira les prompts avec des f-strings : `prompt = f"Contexte : {contexte} Question : {question}"`.

PARTIE 6

Les listes

Définition

Une **liste** est une collection **ordonnée** d'éléments, écrite entre crochets `[ ]` et séparée par des virgules. Chaque élément a une **position (index)** qui commence à **0**.

FICHIER : `exercices/listes.py`

```
1 documents = ["rh.txt", "procedure.pdf", "formation.docx"]
2
3 print(documents[0])
4 print(documents[-1])
5 print(len(documents))
6
7 documents.append("contrat.txt")
8 print(documents)
9
10 for doc in documents:
11     print("Lecture du fichier :", doc)
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>documents = ["rh.txt", "procedure.pdf", "formation.docx"]</code>	Crée une liste de 3 noms de fichiers.
3	<code>print(documents[0])</code>	Affiche l'élément à l'index 0 , c'est-à-dire le premier .
4	<code>print(documents[-1])</code>	L'index -1 désigne le dernier élément.
5	<code>print(len(documents))</code>	len() donne le nombre d'éléments.
7	<code>documents.append("contrat.txt")</code>	append() ajoute un élément à la fin de la liste.
8	<code>print(documents)</code>	Affiche toute la liste.
10	<code>for doc in documents:</code>	Une boucle for : pour chaque élément de la liste, on le met dans la variable <code>doc</code> ... Les deux-points <code>:</code> sont obligatoires.
11	<code>print("Lecture du fichier :", doc)</code>	...et on exécute les lignes indentées (décalées de 4 espaces). <code>print</code> peut afficher plusieurs valeurs séparées par des virgules.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python exercices/listes.py
```

RÉSULTAT OBTENU

```
rh.txt
formation.docx
3
['rh.txt', 'procedure.pdf', 'formation.docx', 'contrat.txt']
Lecture du fichier : rh.txt
Lecture du fichier : procedure.pdf
Lecture du fichier : formation.docx
Lecture du fichier : contrat.txt
```

Attention

L'**indentation** (le décalage de 4 espaces) n'est pas décorative en Python : elle indique quelles lignes appartiennent à la boucle. Une mauvaise indentation provoque l'erreur `IndentationError`.

Lien avec le projet

Au Jour 2, le RAG découpe les documents en **liste de morceaux (chunks)**, et ChromaDB renvoie une **liste** des morceaux les plus pertinents.

PARTIE 7

Les dictionnaires

Définition

Un **dictionnaire** stocke des paires **clé → valeur**, entre accolades `{ }`. On accède à une valeur par sa **clé** (et non par une position).

Analogie simple

C'est comme un **vrai dictionnaire** : on cherche un mot (la clé) pour trouver sa définition (la valeur). Ou une **fiche employé** : « nom » → « Sami ».

FICHIER : `exercices/dictionnaires.py`

```
1 employe = {
2     "nom": "Sami",
3     "poste": "Technicien",
4     "conges": 30,
5 }
6
7 print(employe["nom"])
8 print(employe.get("salaire", "non renseigné"))
9
10 employe["service"] = "Maintenance"
11 print(employe)
12
13 for cle, valeur in employe.items():
14     print(cle, "→", valeur)
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>employe = {</code>	Ouvre un dictionnaire nommé employe .
2	<code>"nom": "Sami",</code>	Clé "nom" → valeur "Sami". Clé et valeur sont séparées par <code>:</code> .
3	<code>"poste": "Technicien",</code>	Chaque paire est séparée par une virgule.
4	<code>"conges": 30,</code>	Une valeur peut être de n'importe quel type (ici un nombre).
5	<code>}</code>	Ferme le dictionnaire.
7	<code>print(employe["nom"])</code>	Récupère la valeur associée à la clé "nom".
8	<code>print(employe.get("salaire", "non renseigné"))</code>	get() cherche une clé ; si elle n'existe pas, renvoie la valeur par défaut au lieu de provoquer une erreur.
10	<code>employe["service"] = "Maintenance"</code>	Ajoute une nouvelle paire clé → valeur.
11	<code>print(employe)</code>	Affiche tout le dictionnaire.

Ligne	Instruction	Explication
13	<code>for cle, valeur in employe.items():</code>	items() donne chaque paire ; la boucle récupère la clé et la valeur.
14	<code>print(cle, "→", valeur)</code>	Affiche chaque paire.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)
<code>python exercices/dictionnaires.py</code>

RÉSULTAT OBTENU
<pre> Sami non renseigné {'nom': 'Sami', 'poste': 'Technicien', 'conges': 30, 'service': 'Maintenance'} nom → Sami poste → Technicien conges → 30 service → Maintenance </pre>

Pourquoi les dictionnaires sont essentiels pour l'IA ?

Quand on interroge Ollama (ou n'importe quelle API d'IA), la réponse arrive au format **JSON**, que Python transforme en **dictionnaire**. Voici à quoi ressemble (en simplifié) la réponse d'Ollama :

FICHER : <code>exercices/reponse_ollama.py</code>
<pre> 1 reponse = { 2 "model": "llama3.2", 3 "response": "Bonjour ! Comment puis-je vous aider ?", 4 "done": True, 5 } 6 print(reponse["response"]) </pre>

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>reponse = {</code>	Dictionnaire qui imite la réponse renvoyée par Ollama.
2	<code>"model": "llama3.2",</code>	Le modèle utilisé.
3	<code>"response": "Bonjour ! Comment puis-je vous aider ?",</code>	Le texte généré par le LLM : c'est ce qui nous intéresse.
4	<code>"done": True,</code>	Indique que la génération est terminée.
6	<code>print(reponse["response"])</code>	On extrait uniquement le texte de la réponse.

RÉSULTAT OBTENU

Bonjour ! Comment puis-je vous aider ?

Lien avec le projet

Dans `llm.py`, on écrira exactement : `reponse.json()["response"]` pour récupérer la réponse du modèle.

PARTIE 8

Les conditions

Définition

Une **condition** permet d'exécuter un bloc de code **seulement si** une situation est vraie : `if` (si), `elif` (sinon si), `else` (sinon).

Opérateur	Signification	Exemple	Résultat
<code>==</code>	est égal à	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	est différent de	<code>5 != 3</code>	<code>True</code>
<code>></code> <code><</code> <code>>=</code> <code><=</code>	comparaisons	<code>10 > 3</code>	<code>True</code>
<code>in</code>	est contenu dans	<code>"calcul" in "calcul 2+2"</code>	<code>True</code>
<code>and</code> / <code>or</code> / <code>not</code>	et / ou / non	<code>True and False</code>	<code>False</code>

FICHIER : exercices/conditions.py

```

1 question = "calcul 12 * 350"
2
3 if question == "Bonjour":
4     print("Route : salutation")
5 elif "calcul" in question:
6     print("Route : calculatrice")
7 else:
8     print("Route : recherche dans les documents")

```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>question = "calcul 12 * 350"</code>	La question posée à l'assistant.
3	<code>if question == "Bonjour":</code>	Si la question est exactement « Bonjour »... (attention : <code>==</code> compare, <code>=</code> affecte).
4	<code>print("Route : salutation")</code>	...on exécute ce bloc indenté.
5	<code>elif "calcul" in question:</code>	Sinon, si le mot « calcul » se trouve dans la question...
6	<code>print("Route : calculatrice")</code>	...on choisit la calculatrice.
7	<code>else:</code>	Sinon (aucune condition vraie)...
8	<code>print("Route : recherche dans les documents")</code>	...on cherchera dans les documents.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python exercices/conditions.py
```

RÉSULTAT OBTENU

Route : calculatrice

À tester

Changez la ligne 1 en `question = "Bonjour"` → résultat : **Route : salutation.**

Puis en `question = "Combien de jours de congé ?"` → résultat : **Route : recherche dans les documents.**

Lien avec le projet

Ce choix de « route » est **exactement** ce que fera notre agent LangGraph au Jour 2 (le *routing*) : salutation, calculatrice ou recherche dans les documents.

PARTIE 9

Les boucles

9.1 La boucle for (répéter un nombre connu de fois)

FICHIER : exercices/boucle_for.py

```
1 for i in range(1, 4):
2     print("Tentative numéro", i)
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>for i in range(1, 4):</code>	<code>range(1, 4)</code> produit les nombres 1, 2, 3 (le 4 est exclu).
2	<code>print("Tentative numéro", i)</code>	Le bloc est exécuté une fois pour chaque nombre.

RÉSULTAT OBTENU

```
Tentative numéro 1
Tentative numéro 2
Tentative numéro 3
```

9.2 La boucle while (répéter tant que...)

La boucle `while True` tourne à l'infini jusqu'à ce qu'on l'arrête avec `break`. C'est la base de **toute conversation avec un chatbot** : on pose une question, il répond, on recommence.

FICHIER : exercices/boucle_while.py

```
1 while True:
2     texte = input("Vous : ")
3     if texte == "quit":
4         print("Fin de la conversation.")
5         break
6     print("Vous avez écrit :", texte)
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>while True:</code>	Boucle infinie : elle recommence sans cesse.
2	<code>texte = input("Vous : ")</code>	<code>input()</code> affiche « Vous : » et attend que l'utilisateur tape un texte puis Entrée. Le texte est stocké dans <code>texte</code> .
3	<code>if texte == "quit":</code>	Si l'utilisateur tape « quit »...
5	<code>break</code>	... break arrête immédiatement la boucle.
6	<code>print("Vous avez écrit :", texte)</code>	Sinon, on affiche ce qu'il a écrit, et la boucle recommence.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python exercices/boucle_while.py
```

RÉSULTAT OBTENU (exemple de saisie)

```
Vous : Bonjour  
Vous avez écrit : Bonjour  
Vous : Ça marche !  
Vous avez écrit : Ça marche !  
Vous : quit  
Fin de la conversation.
```

Attention

Si un programme tourne sans fin et ne répond plus, arrêtez-le avec **Ctrl + C** dans le terminal.

PARTIE 10

Les fonctions

Définition

Une **fonction** est un **bloc de code réutilisable** auquel on donne un nom. On la définit avec `def`, elle reçoit des **paramètres** et renvoie un résultat avec `return`.

Analogie simple

Une fonction, c'est une **machine à café** : on met des ingrédients (paramètres), on appuie sur le bouton (appel), et on récupère un café (le `return`).

FICHER : `exercices/fonctions.py`

```
1 def saluer(prenom):
2     return f"Bonjour {prenom}, bienvenue !"
3
4 def calculer_prix(nb_participants, nb_jours, prix_jour=350):
5     return nb_participants * nb_jours * prix_jour
6
7 message = saluer("Amira")
8 print(message)
9 print(calculer_prix(3, 3))
10 print(calculer_prix(3, 3, prix_jour=400))
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>def saluer(prenom):</code>	Définit une fonction saluer avec un paramètre prenom . Les <code>:</code> et l'indentation sont obligatoires.
2	<code>return f"Bonjour {prenom}, bienvenue !"</code>	return renvoie le résultat à celui qui a appelé la fonction.
4	<code>def calculer_prix(nb_participants, nb_jours, prix_jour=350):</code>	Fonction à 3 paramètres. <code>prix_jour=350</code> est une valeur par défaut : utilisée si on ne la précise pas.
5	<code>return nb_participants * nb_jours * prix_jour</code>	Calcule et renvoie le prix total.
7	<code>message = saluer("Amira")</code>	Appel de la fonction : <code>prenom</code> reçoit « Amira ». Le résultat est stocké dans <code>message</code> .
9	<code>print(calculer_prix(3, 3))</code>	3 participants × 3 jours × 350 (valeur par défaut).
10	<code>print(calculer_prix(3, 3, prix_jour=400))</code>	On remplace la valeur par défaut par 400.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)
<code>python exercices/fonctions.py</code>

RÉSULTAT OBTENU
Bonjour Amira, bienvenue ! 3150 3600

Aperçu : la fonction centrale du projet

Voici la fonction que nous allons écrire au **TP 3**. Pour l'instant, elle renvoie une réponse **simulée** ; au TP 3, on remplacera la ligne 3 par un vrai appel à Ollama.

FICHIER : exercices/aperçu_llm.py
<pre> 1 def demander_llm(question): 2 # Au TP 3, on appellera Ollama ici 3 return f"(Réponse simulée) Vous avez demandé : {question}" 4 5 print(demander_llm("Qu'est-ce qu'un LLM ?")) </pre>

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>def demander_llm(question):</code>	La fonction qui posera une question au LLM.
2	<code># Au TP 3, on appellera Ollama ici</code>	Commentaire : rappel de ce qu'on fera plus tard.
3	<code>return f"(Réponse simulée) Vous avez demandé : {question}"</code>	Réponse provisoire pour tester.
5	<code>print(demander_llm("Qu'est-ce qu'un LLM ?"))</code>	Appel et affichage du résultat.

RÉSULTAT OBTENU
(Réponse simulée) Vous avez demandé : Qu'est-ce qu'un LLM ?

Lien avec le projet

Tout le projet est organisé en fonctions : `demander_llm()`, `lire_document()`, `rechercher()`, `repondre()`... Chaque fichier **importe** les fonctions du précédent.

PARTIE 11

Lire un fichier

Notre assistant doit lire les **documents de l'entreprise**. Créez le dossier **documents** et, dedans, le fichier **rh.txt** :

FICHIER : documents/rh.txt
Règlement intérieur - Service RH Horaires : du lundi au vendredi, de 8h à 17h. Congés : 30 jours par an. Contact : rh@entreprise.tn

11.1 Lire tout le fichier

FICHIER : exercices/lire_fichier.py
<pre> 1 with open("documents/rh.txt", "r", encoding="utf-8") as fichier: 2 contenu = fichier.read() 3 4 print(contenu) 5 print("Nombre de caractères :", len(contenu)) </pre>

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>with open("documents/rh.txt", "r", encoding="utf-8") as fichier:</code>	open() ouvre le fichier : "r" = lecture (<i>read</i>), <code>encoding="utf-8"</code> = lire correctement les accents. with referme automatiquement le fichier à la fin du bloc.
2	<code>contenu = fichier.read()</code>	read() lit tout le fichier et le met dans une seule chaîne de texte.
4	<code>print(contenu)</code>	Affiche le contenu.
5	<code>print("Nombre de caractères :", len(contenu))</code>	len() fonctionne aussi sur le texte : il compte les caractères.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)
<code>python exercices/lire_fichier.py</code>

RÉSULTAT OBTENU

Règlement intérieur - Service RH
 Horaires : du lundi au vendredi, de 8h à 17h.
 Congés : 30 jours par an.
 Contact : rh@entreprise.tn

Nombre de caractères : 132

11.2 Lire ligne par ligne

FICHIER : exercices/lire_lignes.py

```
1 with open("documents/rh.txt", "r", encoding="utf-8") as fichier:
2     lignes = fichier.readlines()
3
4 print("Nombre de lignes :", len(lignes))
5 for numero, ligne in enumerate(lignes, start=1):
6     print(numero, "-", ligne.strip())
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>with open("documents/rh.txt", "r", encoding="utf-8") as fichier:</code>	Même ouverture que précédemment.
2	<code>lignes = fichier.readlines()</code>	readlines() renvoie une liste : un élément par ligne.
5	<code>for numero, ligne in enumerate(lignes, start=1):</code>	enumerate() donne à la fois le numéro et la ligne ; start=1 commence à 1.
6	<code>print(numero, "-", ligne.strip())</code>	strip() supprime le retour à la ligne et les espaces en trop au début et à la fin.

RÉSULTAT OBTENU

Nombre de lignes : 4
 1 - Règlement intérieur - Service RH
 2 - Horaires : du lundi au vendredi, de 8h à 17h.
 3 - Congés : 30 jours par an.
 4 - Contact : rh@entreprise.tn

Attention

Sans `encoding="utf-8"`, les accents peuvent s'afficher mal (*RÃˆglement*) sous Windows.

FileNotFoundError : le chemin est faux ou le terminal n'est pas dans le dossier du projet.

Lien avec le projet

Au Jour 2, on lira de la même façon les fichiers PDF (`pypdf`) et Word (`python-docx`), puis on découpera le texte pour le RAG.

PARTIE 12

Modules, API et gestion des erreurs

12.1 Importer un module

Définition

Un **module** est un fichier de code prêt à l'emploi. On l'utilise avec `import`. Certains sont fournis avec Python (`math`, `datetime`), d'autres s'installent avec pip (`requests`, `fastapi`).

FICHIER : `exercices/modules.py`

```
1 import math
2 from datetime import date
3
4 print(math.sqrt(16))
5 print("Date du jour :", date.today())
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>import math</code>	Importe tout le module <code>math</code> .
2	<code>from datetime import date</code>	Importe uniquement <code>date</code> depuis le module <code>datetime</code> .
4	<code>print(math.sqrt(16))</code>	On utilise une fonction du module avec <code>module.fonction()</code> : racine carrée.
5	<code>print("Date du jour :", date.today())</code>	Affiche la date du jour (format année-mois-jour).

RÉSULTAT OBTENU (la date dépend du jour)

```
4.0
Date du jour : 2026-10-12
```

Lien avec le projet

Dans le projet : `from llm import demander_llm` permettra à `rag.py` d'utiliser la fonction écrite dans `llm.py`. **Nos propres fichiers deviennent des modules.**

12.2 Appeler une API avec requests

Définition

Une **API** est une « porte d'entrée » qui permet à un programme de demander quelque chose à un autre programme via Internet ou le réseau. On envoie une **requête**, on reçoit une **réponse** (souvent en JSON).

Analogie simple

Au restaurant, vous ne rentrez pas en cuisine : vous passez commande au **serveur** (l'API), qui vous rapporte le plat (la réponse).

Exemple avec une API publique de test (connexion Internet nécessaire) :

FICHER : exercices/api_get.py

```
1 import requests
2
3 url = "https://jsonplaceholder.typicode.com/todos/1"
4 reponse = requests.get(url)
5
6 print(reponse.status_code)
7 donnees = reponse.json()
8 print(donnees)
9 print(donnees["title"])
```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>import requests</code>	Importe la bibliothèque installée avec <code>pip install requests</code> .
3	<code>url = "https://jsonplaceholder.typicode.com/todos/1"</code>	L'adresse de l'API à interroger.
4	<code>reponse = requests.get(url)</code>	get() envoie une requête de lecture et récupère la réponse.
6	<code>print(reponse.status_code)</code>	Code de statut HTTP : 200 = succès, 404 = introuvable, 500 = erreur serveur.
7	<code>donnees = reponse.json()</code>	json() transforme la réponse JSON en dictionnaire Python.
9	<code>print(donnees["title"])</code>	On lit une valeur du dictionnaire, comme vu en partie 7.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python exercices/api_get.py
```

RÉSULTAT OBTENU

```
200
{'userId': 1, 'id': 1, 'title': 'delectus aut autem', 'completed': False}
delectus aut autem
```

Aperçu : envoyer une question à Ollama (TP 3)

Pour poser une question à un LLM, on utilise **post()** : on **envoie** des données (le modèle et le prompt). Ce code fonctionnera au TP 3, une fois Ollama installé :

FICHIER : apercu_tp3.py (ne pas exécuter maintenant)

```
1 import requests
2
3 url = "http://localhost:11434/api/generate"
4 donnees = {
5     "model": "llama3.2",
6     "prompt": "Explique l'IA en une phrase.",
7     "stream": False,
8 }
9 reponse = requests.post(url, json=donnees)
10 print(reponse.json()["response"])
```

Explication ligne par ligne :

Ligne	Instruction	Explication
3	<code>url = "http://localhost:11434/api/generate"</code>	localhost = votre propre ordinateur ; 11434 = le port d'Ollama.
4	<code>donnees = {</code>	Les données envoyées sont un dictionnaire .
5	<code>"model": "llama3.2",</code>	Le modèle à utiliser.
6	<code>"prompt": "Explique l'IA en une phrase.",</code>	La question (le prompt).
7	<code>"stream": False,</code>	False : on veut la réponse complète d'un coup (et non mot par mot).
9	<code>reponse = requests.post(url, json=donnees)</code>	post() envoie le dictionnaire au format JSON à Ollama.
10	<code>print(reponse.json()["response"])</code>	On extrait le texte généré, exactement comme en partie 7.

12.3 Gérer les erreurs avec try / except

Définition

try / except permet d'**essayer** un code et, s'il provoque une erreur, d'exécuter un plan B au lieu de faire planter le programme.

FICHER : exercices/erreurs.py

```

1 import requests
2
3 try:
4     reponse = requests.get("http://localhost:11434", timeout=5)
5     print("Ollama répond :", reponse.text)
6 except requests.exceptions.ConnectionError:
7     print("Erreur : Ollama n'est pas démarré.")

```

Explication ligne par ligne :

Ligne	Instruction	Explication
3	<code>try:</code>	Essaie d'exécuter le bloc suivant...
4	<code>reponse = requests.get("http://localhost:11434", timeout=5)</code>	On tente de contacter Ollama. <code>timeout=5</code> : on n'attend pas plus de 5 secondes.
5	<code>print("Ollama répond :", reponse.text)</code>	Si ça marche, on affiche la réponse (<code>.text</code> = réponse en texte brut).
6	<code>except requests.exceptions.ConnectionError:</code>	... si une erreur de connexion survient (Ollama éteint ou pas installé)...
7	<code>print("Erreur : Ollama n'est pas démarré.")</code>	...on affiche un message clair au lieu d'un plantage.

RÉSULTAT OBTENU (aujourd'hui, Ollama n'est pas encore installé)

Erreur : Ollama n'est pas démarré.

RÉSULTAT OBTENU (après le TP 2, Ollama démarré)

Ollama répond : Ollama is running

Lien avec le projet

Dans le projet, chaque appel au LLM sera protégé par un `try / except`, pour que l'assistant affiche un message clair au lieu de planter.

PARTIE 13

Mini-projet : le mini-assistant RH (version 0)

Objectifs

- Réunir **toutes** les notions de ce document dans un seul programme
- Créer la **version 0** de notre Assistant IA d'Entreprise : un assistant **sans IA**, basé sur des règles
- Comprendre ses limites... que le LLM viendra résoudre au TP 3

Créez le fichier **mini_assistant.py** à la racine du projet (pas dans `exercices`) :

FICHIER : mini_assistant.py

```

1  faq = {
2      "horaires": "Nous sommes ouverts du lundi au vendredi, de 8h à 17h.",
3      "congé": "Chaque employé a droit à 30 jours de congés par an.",
4      "contact": "Écrivez à rh@entreprise.tn pour toute question RH.",
5  }
6
7
8  def calculer(expression):
9      try:
10         return f"Résultat : {eval(expression)}"
11     except Exception:
12         return "Je n'ai pas compris le calcul."
13
14
15  def repondre(question):
16      question = question.lower()
17
18      if question in ["bonjour", "salut", "salam"]:
19          return "Bonjour ! Je suis votre assistant. Posez-moi une question."
20      elif question.startswith("calcul"):
21          expression = question.replace("calcul", "").strip()
22          return calculer(expression)
23
24      for mot_cle, reponse in faq.items():
25          if mot_cle in question:
26              return reponse
27
28      return "Désolé, je ne connais pas encore la réponse."
29
30
31  print("=== Mini-assistant RH (tapez 'quit' pour sortir) ===")
32  while True:
33      question = input("Vous : ")
34      if question.lower() == "quit":
35          print("Assistant : Au revoir !")
36          break
37      print("Assistant :", repondre(question))

```

Explication ligne par ligne :

Ligne	Instruction	Explication
1	<code>faq = {</code>	Un dictionnaire : mot-clé → réponse. C'est la « mémoire » de notre assistant.
3	<code>"congé": "Chaque employé a droit à 30 jours de congés par an.",</code>	« congé » détectera aussi « congés » (car « congé » est contenu dans « congés »).
8	<code>def calculer(expression):</code>	Fonction calculatrice : reçoit un calcul sous forme de texte.
9	<code>try:</code>	On essaie de calculer...
10	<code>return f"Résultat : {eval(expression)}"</code>	eval() calcule une expression écrite en texte (ex. « 12 * 350 »).
11	<code>except Exception:</code>	...si le calcul est invalide (ex. « 12 * abc »)...
12	<code>return "Je n'ai pas compris le calcul."</code>	...on renvoie un message au lieu de planter.
15	<code>def repondre(question):</code>	La fonction principale : reçoit une question, renvoie une réponse.
16	<code>question = question.lower()</code>	lower() met en minuscules : « Bonjour » et « BONJOUR » deviennent identiques.
18	<code>if question in ["bonjour", "salut", "salam"]:</code>	Route 1 — salutation : la question est-elle dans la liste ?
20	<code>elif question.startswith("calcul"):</code>	Route 2 — calculatrice : la question commence-t-elle par « calcul » ?
21	<code>expression = question.replace("calcul", "").strip()</code>	On retire le mot « calcul » et les espaces : il reste « 12 * 350 ».
22	<code>return calculer(expression)</code>	On appelle la fonction calculatrice.
24	<code>for mot_cle, reponse in faq.items():</code>	Route 3 — documents : on parcourt chaque mot-clé de la FAQ...
25	<code>if mot_cle in question:</code>	...si le mot-clé est présent dans la question...
26	<code>return reponse</code>	...on renvoie la réponse associée (et la fonction s'arrête).
28	<code>return "Désolé, je ne connais pas encore la réponse."</code>	Aucune route trouvée : réponse par défaut.
31	<code>print("=== Mini-assistant RH (tapez 'quit' pour sortir) ===")</code>	Message d'accueil.
32	<code>while True:</code>	Boucle de conversation .
33	<code>question = input("Vous : ")</code>	On attend la question de l'utilisateur.
34	<code>if question.lower() == "quit":</code>	Commande pour quitter.
36	<code>break</code>	Sortie de la boucle.
37	<code>print("Assistant :", repondre(question))</code>	On appelle <code>repondre()</code> et on affiche le résultat.

Exécution : dans le terminal de VS Code, tapez :

COMMANDE (TERMINAL)

```
python mini_assistant.py
```

RÉSULTAT OBTENU (exemple de conversation)

```
=== Mini-assistant RH (tapez 'quit' pour sortir) ===
Vous : Bonjour
Assistant : Bonjour ! Je suis votre assistant. Posez-moi une question.
Vous : Quels sont les horaires ?
Assistant : Nous sommes ouverts du lundi au vendredi, de 8h à 17h.
Vous : calcul 12 * 350
Assistant : Résultat : 4200
Vous : Combien de jours de congé ?
Assistant : Chaque employé a droit à 30 jours de congés par an.
Vous : Quelle est la météo ?
Assistant : Désolé, je ne connais pas encore la réponse.
Vous : quit
Assistant : Au revoir !
```

Sécurité

eval() exécute n'importe quel code Python : c'est **dangereux** si n'importe qui peut écrire dans l'assistant. On l'utilise ici uniquement pour apprendre ; dans une application réelle, on utilise une calculatrice sécurisée.

13.1 Les limites de la version 0

Question posée	Réponse	Pourquoi ?
« Bonjour ! »	Désolé...	« bonjour ! » (avec le point d'exclamation) n'est pas exactement dans la liste.
« À quelle heure ouvrez-vous ? »	Désolé...	Le mot-clé « horaires » n'apparaît pas : l'assistant ne comprend pas le sens .
« Combien coûte la formation ? »	Désolé...	L'information n'est pas dans la FAQ.

Ce que vont apporter les prochaines étapes

Notre assistant actuel ne fait que **chercher des mots exacts**. Il ne comprend pas le langage. Au **TP 3**, on remplacera ces règles par un **LLM** (Ollama) qui comprend le sens ; au **Jour 2**, le **RAG** lui permettra de lire les vrais documents. C'est exactement le passage de l'**IA symbolique** à l'**IA générative** vu ce matin !

PARTIE 14

Exercices

Créez chaque exercice dans le dossier **exercices/**. Les corrigés sont à la fin de chaque exercice.

Exercice 1 — Variables et f-string

Créez 3 variables `prenom`, `age` et `ville`, puis affichez la phrase : *Je m'appelle ..., j'ai ... ans et j'habite à ...*

CORRIGÉ

```
prenom = "Amira"
age = 24
ville = "Monastir"
print(f"Je m'appelle {prenom}, j'ai {age} ans et j'habite à {ville}.")
```

RÉSULTAT OBTENU

Je m'appelle Amira, j'ai 24 ans et j'habite à Monastir.

Exercice 2 — Listes

Avec la liste `notes = [12, 15, 9, 18]` : affichez la moyenne et la meilleure note, puis ajoutez la note 14 et affichez le nombre de notes.

CORRIGÉ

```
notes = [12, 15, 9, 18]
moyenne = sum(notes) / len(notes)    # sum() additionne tous les éléments
print("Moyenne :", moyenne)
print("Meilleure note :", max(notes)) # max() renvoie le plus grand
notes.append(14)
print("Nombre de notes :", len(notes))
```

RÉSULTAT OBTENU

Moyenne : 13.5
Meilleure note : 18
Nombre de notes : 5

Exercice 3 — Dictionnaires

Créez un dictionnaire `produit` (`nom` : « Clavier », `prix_ht` : 100, `stock` : 25) et affichez le prix TTC (TVA 19 %) avec 2 décimales.

CORRIGÉ

```
produit = {"nom": "Clavier", "prix_ht": 100, "stock": 25}
prix_ttc = produit["prix_ht"] * 1.19
print(f"{produit['nom']} : {prix_ttc:.2f} DT TTC")    # :.2f = 2 décimales
```

RÉSULTAT OBTENU

Clavier : 119.00 DT TTC

Exercice 4 — Fonction et condition

Écrivez une fonction `est_salutation(texte)` qui renvoie `True` si le texte est « bonjour », « salut », « salam » ou « bonsoir » (quelle que soit la casse), sinon `False`.

CORRIGÉ

```
def est_salutation(texte):
    return texte.lower() in ["bonjour", "salut", "salam", "bonsoir"]

print(est_salutation("Bonjour"))
print(est_salutation("Quel est mon solde ?"))
```

RÉSULTAT OBTENU

```
True
False
```

Exercice 5 — Fichier

Lisez `documents/rh.txt` et affichez **uniquement** la ligne qui contient le mot « Contact ».

CORRIGÉ

```
with open("documents/rh.txt", "r", encoding="utf-8") as fichier:
    for ligne in fichier:          # on peut parcourir un fichier ligne par ligne
        if "Contact" in ligne:
            print(ligne.strip())
```

RÉSULTAT OBTENU

Contact : rh@entreprise.tn

Défi (profil avancé) — Améliorer le mini-assistant

- Faire reconnaître « Bonjour ! » en supprimant la ponctuation (indice : `question.strip("!?.").`).
- Ajouter une entrée « salaire » dans la FAQ.
- Compter le nombre de questions posées et l'afficher quand l'utilisateur tape « quit ».

PARTIE 15

Récapitulatif

Ce que nous réutiliserons dans le projet

Notion	Syntaxe clé	Où dans le projet ?
Variables, f-string	<code>f"Question : {question}"</code>	Construire les prompts (<code>llm.py</code> , <code>rag.py</code>)
Listes	<code>[]</code> , <code>append()</code> , <code>for</code>	Morceaux de documents (chunks), résultats de recherche
Dictionnaires	<code>{ "cle": valeur }</code> , <code>["cle"]</code>	Réponses JSON d'Ollama, état de l'agent (State)
Conditions	<code>if / elif / else</code>	Routage de l'agent (Jour 2)
Boucle while	<code>while True + break</code>	Conversation en ligne de commande
Fonctions	<code>def ... return</code>	Toutes les briques : <code>demander_llm()</code> , <code>repondre()</code> ...
Fichiers	<code>with open(..., encoding="utf-8")</code>	Lecture des documents (Jour 2)
Modules	<code>import</code> , <code>from ... import</code>	Relier <code>llm.py</code> → <code>rag.py</code> → <code>agent.py</code> → <code>main.py</code>
API	<code>requests.get / post</code>	Appeler Ollama (TP 3), tester FastAPI (Jour 3)
Erreurs	<code>try / except</code>	Ollama éteint, fichier absent, mauvaise saisie

Mini-glossaire Python

Terme	Définition
Script	Fichier <code>.py</code> contenant un programme Python.
Terminal	Fenêtre où l'on tape des commandes (PowerShell sous Windows).
venv	Environnement virtuel : Python isolé, propre au projet.
pip	Outil qui installe les bibliothèques Python.
Indentation	Décalage de 4 espaces qui définit les blocs de code (après <code>:</code>).
Paramètre	Valeur reçue par une fonction.
return	Mot-clé qui renvoie le résultat d'une fonction.
JSON	Format texte d'échange de données entre programmes ; devient un dictionnaire en Python.
API	Porte d'entrée permettant à un programme d'interroger un autre programme.
localhost	Votre propre ordinateur (adresse 127.0.0.1).

Et ensuite ?

Prochaine étape : **TP 1 — Mon premier modèle de A à Z** avec scikit-learn, puis **TP 2 et 3** : installer Ollama et créer `llm.py`, le premier vrai fichier de notre assistant.

— *Fin du document* —